

Nama: Hans Malvin Djojo

<https://github.com/hansmalvin/LastMile>

Environment Setup

Create a New React Application:

Open your terminal and use Create React App to set up a new project.

```
bash
npx create-react-app api-integration-lab
cd api-integration-lab
```

Install Required Packages (Optional):

For this lab, we don't strictly need additional packages beyond what Create React App provides. However, if you wish to use a library like axios for API calls or a UI library like material-ui or bootstrap, install them now. For example, to install axios:

```
bash
npm install axios
```

Set up a Mock API (Optional):

For the purpose of this lab, we'll use a simple online JSON placeholder API: <https://jsonplaceholder.typicode.com/users>. This API provides sample user data. Alternatively, you could set up a local mock API using tools like json-server if you prefer to work offline. To install json-server:

```
bash
npm install -g json-server
```

Create a db.json file with sample data:

```
json
{
  "users": [
    { "id": 1, "name": "John Doe", "email": "john.doe@example.com" },
    { "id": 2, "name": "Jane Smith", "email": "jane.smith@example.com" }
  ]
}
```

Start the mock server:

```
bash
json-server --watch db.json --port 3001
```

You would then use `http://localhost:3001/users` as your API endpoint. For simplicity, we'll stick to `https://jsonplaceholder.typicode.com/users` in the following instructions.

Project Structure:

Your project structure should look like this:

```
api-integration-lab/
├── node_modules/
├── public/
├── src/
│   ├── App.css
│   ├── App.js
│   ├── App.test.js
│   ├── index.css
│   ├── index.js
│   ├── logo.svg
│   └── setupTests.js
├── package.json
├── README.md
└── ...
```

Step-by-Step Instructions

Step 1: Create a User Component

Create a new file named `User.js` in the `src` directory. This component will display individual user information.

```
javascript
// src/User.js
import React from 'react';
function User({ user }) {
  return (

    {user.name}
    Email: {user.email}

  );
}
export default User;
```

This component takes a user object as a prop and renders the user's name and email. This separation of concerns makes the code more modular and readable.

Step 2: Modify the App Component to Fetch Data

Open src/App.js and modify it to fetch data from the API.

```
javascript
// src/App.js
import React, { useState, useEffect } from 'react';
import User from './User';
import './App.css';
function App() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await fetch('https://jsonplaceholder.typicode.com/users');
        if (!response.ok) {
          throw new Error(HTTP error! status: ${response.status});
        }
        const data = await response.json();
        setUsers(data);
      } catch (e) {
        setError(e);
      } finally {
        setLoading(false);
      }
    };

    fetchData();
  }, []); // Empty dependency array ensures this effect runs only once on mount

  if (loading) {
    return

    Loading users...
    ;
  }
  if (error) {
    return

    Error: {error.message}
    ;
  }
}
```

```

return (
  <User List
    {users.map(user => (
      <User
        {user}
      )}
    )}
  </User List>
);
}
export default App;

```

Explanation:

Import necessary modules: Imports `useState` and `useEffect` from `React`, the `User` component, and the `App.css` stylesheet.

State variables: Uses `useState` to manage the `users` array, the loading state (initially `true`), and the error state (initially `null`).

`useEffect` hook: The `useEffect` hook is used to perform side effects in functional components. The empty dependency array `[]` ensures that the effect runs only once, similar to `componentDidMount` in class components.

`fetchData` function: This asynchronous function fetches data from the API endpoint using the `fetch` API. It handles potential errors using a `try...catch...finally` block.

Error handling: If the `response.ok` property is `false` (indicating an HTTP error), an error is thrown. The `catch` block updates the error state with the error message.

Loading and Error indicators: While data is being fetched, a "Loading users..." message is displayed. If an error occurs, an error message is displayed.

Rendering the `User` components: Once the data is fetched successfully, the `users` array is mapped to an array of `User` components, each displaying information about a single user. The `key` prop is important for `React` to efficiently update the list.

Step 3: Style the App (Optional)

Open `src/App.css` and add some basic styling.

```

css
/ src/App.css /
.App {
  text-align: center;
  padding: 20px;
}

```

```
}  
.App h1 {  
color: #333;  
}  
  
.User {  
margin: 10px;  
padding: 10px;  
border: 1px solid #ccc;  
border-radius: 5px;  
}
```

This adds some basic styling to center the content, add padding, and style the user component with a border.

Step 4: Run the Application

Start the React development server.

```
bash  
npm start
```

Open your browser and navigate to <http://localhost:3000>. You should see a list of users fetched from the API.

Validation & Expected Results

After running the application, you should see a list of users displayed on the page. Each user should have their name and email address displayed. If there are any errors, an error message should be displayed instead. If the data is still loading, a "Loading users..." message should be displayed.

Sample Output:

The browser should display something similar to:

```
User List  
John Doe  
Email: john.doe@example.com  
  
Jane Smith  
Email: jane.smith@example.com  
  
... (other users)
```

If you are using the json-server with the sample db.json provided in the Environment Setup, you should see the two users defined in that file. If using the placeholder API, you will see 10 users.

Troubleshooting Tips

Answer:

Using mock API:

